



TECHNICAL HANDOVER DOCUMENT

Glass Wall Systems CMS

Corporate Website & Content Management System — Architecture, Functionality & Developer Guide

Laravel 12 · PHP 8.2 · MySQL

Prepared for developer onboarding & project handover

Version 1.0 — October 2026

Table of Contents

1. Project Overview	
2. Technology Stack & Environment	
3. Project Structure	
4. Architecture & Request Lifecycle	
5. Authentication & Authorization (RBAC)	
6. Data Model & Database	
7. Admin CMS (Backend)	
8. Public Website (Frontend)	
9. Feature Deep-Dives (SEO, Disclosures, Stock Exchange, Forms, About slider, IPO/DRHP)	
10. File Uploads & Storage	
11. 301 Redirects & .htaccess	
12. Third-Party Integrations	
13. Seeders & Initial Data	
14. Conventions, Gotchas & Known Issues	
15. Deployment & Maintenance Guide	
16. Common Admin Tasks (How-To)	
17. Appendix — Routes, Permissions & Upload Directories	

1. Project Overview

Glass Wall Systems CMS is a custom-built Laravel application that powers the corporate website of *Glass Wall Systems (India) Limited* — a façade / glass-wall engineering company. It is a fully **content-managed** site: virtually every piece of text, image, document and listing on the public website is editable from a password-protected admin panel, with a granular role & permission system.

What the system does

- **Public website** — marketing site with Home, About/Overview (Board, Innovation, ESG, Media, Awards), Products, Projects, Infrastructure (Design & Engineering, Facility, Project Management), Investors Relations (IPO/DRHP, Corporate Governance, Annual Reports, Investor Resources, Disclosures, Stock Exchange), Careers and Contact pages.
- **Admin CMS** — one editable module per page/section; repeatable child rows (galleries, counters, documents, tabs); image/video/PDF uploads; rich-text editing.
- **Lead capture** — Contact & Career forms with validation, Google reCAPTCHA v2, database storage and branded email notifications (resume upload for careers).
- **SEO** — per-URL meta-tag manager, sitemap/robots passthrough, 301 redirect map from the old site, structured data (JSON-LD) on the home page.
- **Governance** — full audit trail (Activity Log) of admin changes; role-based access control.

Design philosophy

The architecture follows one consistent pattern: “**singleton page settings + ordered child rows.**” Almost every public page is backed by a single parent record edited in the CMS, with repeatable sub-items stored in child tables (e.g. **Facility** → features / counters / galleries / strengths / testing-images). Understanding this one pattern unlocks ~90% of the codebase.

2. Technology Stack & Environment

Framework	Laravel 12 (Laravel 11/12 bootstrap style — no <code>Http/Kernel.php</code> ; middleware configured in <code>bootstrap/app.php</code>)
Language	PHP 8.2+
Database	MySQL (local DB <code>glasswall_cms</code> ; production <code>glasswal_live_db</code>)
Local dev	XAMPP on Windows; <code>php artisan serve</code> → <code>http://127.0.0.1:8000</code> , or Apache at <code>/glasswall_cms/public</code>
Production	cPanel / LiteSpeed shared hosting; project root = <code>public_html</code> (served via root <code>.htaccess</code> → <code>server.php</code>)
Front-end libs	Bootstrap 5, Swiper (sliders), GSAP + ScrollTrigger (animations, smooth-scroll), Magnific Popup, Odometer (counters), Isotope, jQuery
Admin theme	"Riho" Bootstrap 5 admin template; CKEditor 5 (Classic) + Summernote for rich text; DataTables for lists
Rich text	CKEditor 5 Classic (CDN) on <code>textarea.editor</code> / <code>#editor</code>
Mail	Laravel Mail over SMTP (credentials in <code>.env</code>); failover mailer smtp → log
Captcha	Google reCAPTCHA v2 ("I'm not a robot" checkbox)
Analytics	Google Tag Manager (container <code>GTM-K5CF8RLQ</code>)

Important Blade gotcha: In this project, single-line `@php ... @endphp` blocks compile incorrectly. Always use a multi-line `@php` block, or inline the expression directly in `@if` / `@foreach` .

3. Project Structure

```
glasswall_cms/
├─ app/
│  ├─ Http/Controllers/
│  │  ├─ Backend/          ← 35 admin CMS controllers (one per module)
│  │  ├─ Frontend/
│  │  │  ├─ HomeController.php ← ALL public page rendering
│  │  │  └─ FormController.php ← Contact & Career form handling
│  │  ├─ LoginController.php ← auth + admin dashboard
│  │  ├─ RoleController / UserController / PermissionController
│  │  └─ ActivityLogController.php
│  ├─ Http/Middleware/CheckPermission.php ← 'permission' alias
│  ├─ Models/          ← ~60 Eloquent models (+ Concerns/TracksDeletedBy)
│  ├─ Observers/AuditObserver.php ← audit trail
│  ├─ Support/ActivityLogger.php ← audit writer
│  ├─ Rules/Recaptcha.php ← reCAPTCHA v2 verifier
│  ├─ Mail/BrandedFormMail.php ← branded form emails
│  └─ Providers/AppServiceProvider.php ← observers + view composers
├─ bootstrap/app.php ← middleware aliases, guest/auth redirects
├─ config/          ← services.php (recaptcha), forms.php (recipients), mail.php
├─ database/
│  ├─ migrations/ ← schema (chronological 2026_08 → 2026_10)
│  └─ seeders/    ← RolePermission, SeoMeta, Disclosure, StockExchange (+ data/*.csv)
├─ public/
│  ├─ .htaccess ← stock Laravel front controller
│  ├─ frontend/assets/ ← css (main.css), js (main.js), images
│  ├─ admin/assets/ ← admin theme assets
│  └─ <upload dirs>/ ← about/, product-listings/, facility-uploads/, etc.
├─ resources/views/
│  ├─ frontend/ ← public pages
│  ├─ backend/ ← admin pages (per module folders)
│  └─ components/{frontend,backend}/ ← head, header, sidebar, footer, main-js
├─ routes/web.php ← ALL routes (no api.php used)
└─ .htaccess ← ROOT (public_html) – HTTPS, 301 map, rewrite to public/
```

4. Architecture & Request Lifecycle

There is a single route file (`routes/web.php`) with no URL prefix separating admin from public — they are distinguished by **middleware groups** and **route-name prefixes** (`admin.*` / `frontend.*`).

Three route groups

Group	Middleware	Purpose
Guest auth	<code>guest</code>	Login, register, forgot/reset password
Authenticated admin	<code>auth</code> + per-route <code>permission:*</code>	Dashboard, user/role management, all CMS modules
Frontend (public)	none	The public website

Middleware is wired in `bootstrap/app.php` : `redirectGuestsTo('/login')` , `redirectUsersTo('/dashboard')` , and the alias `'permission' => CheckPermission::class` .

Public request flow

1. Browser hits a public URL (e.g. `/facility`).
2. Route resolves to a `HomeController` method which eager-loads the relevant parent + child records.
3. The Blade view renders; shared components (`head` , `header` , `footer`) receive data via **View Composers** (nav categories, footer contact, SEO meta) so every page automatically has navigation, footer and per-URL SEO without each controller re-fetching them.

The greedy route & ordering rule

`GET /{category:slug}/{project:slug}` is a catch-all that matches *any* two-segment URL (used for project detail pages). Because Laravel matches in declaration order, **every literal two-segment route must be declared BEFORE it** — e.g. `/products/{category}` , `/ipo/disclaimer` , `/disclosures/{row}` . Adding a new literal multi-segment route *after* the greedy one will 404. This is the single most important routing rule in the project.

5. Authentication & Authorization (RBAC)

Authentication

Handled by `LoginController` (custom, not Breeze/Jetstream). Login validates credentials via `Auth::attempt()`, then checks `is_active` (deactivated users are logged out immediately), regenerates the session and redirects to the dashboard. Forgot/reset password uses Laravel's native password broker. Views live under `resources/views/backend/` (login, register, forgot_password, reset_password).

Role & Permission model

Entity	Relationship	Notes
User	<code>belongsTo(Role)</code> via <code>role_id</code>	One role per user (NOT a many-to-many). <code>is_active</code> flag. Methods: <code>hasRole()</code> , <code>isSuperAdmin()</code> , <code>hasPermission()</code> .
Role	<code>belongsToMany(Permission)</code> via <code>permission_role</code>	Has <code>slug</code> , <code>is_protected</code> , <code>is_active</code> . <code>superadmin</code> slug is special.
Permission	<code>belongsToMany(Role)</code>	Has <code>slug</code> (e.g. <code>manage-facility.edit</code>) + <code>module</code> (for grouping in the catalog UI).

The only pivot is `permission_role`. There is **no** `role_user` table — a user's single role is stored in `users.role_id`.

Permission naming convention

Every CMS module's permission slugs follow `<resource-route-name>.<action>` where action ∈ `view | create | edit | delete`. The route name doubles as the permission prefix — this is enforced by the `$cms()` helper (see §7).

```
// Example: the Facility module (route name "manage-facility")
manage-facility.view      manage-facility.create
manage-facility.edit      manage-facility.delete
```

Super Admin bypass

Implemented in code: `User::hasPermission()` returns `true` immediately if `isSuperAdmin()` (role slug === `superadmin`), so a Super Admin passes every `permission:*` gate regardless of pivot rows. Inactive users are always denied. The **Activity Log** section carries no permission middleware — it is gated "Super Admin only" inside the controller.

Where permissions come from

- RolePermissionSeeder** — seeds the *system* permissions (dashboard, roles, users, permissions) and the 3 base roles (`superadmin`, `admin`, `user`), and elevates the first user to Super Admin.
- Migrations** — the per-module CRUD permissions are seeded idempotently by migrations: `add_cms_module_permissions` (the big catalog), `add_seo_module_permissions`, `add_disclosure_permissions`, `add_stock_exchange_permissions`. Each grants its slugs to Super Admin.

To rebuild the full permission set you must run **both** the migrations and `RolePermissionSeeder`. A freshly self-registered user has *no* role and fails every gate until an admin assigns one.

6. Data Model & Database

~60 Eloquent models in `app/Models/`. Recurring conventions:

- **Audit columns** `created_by` / `updated_by` / `deleted_by` on most content tables (app-managed, not FK-constrained).
- **Soft deletes** + the `TracksDeletedBy` trait (stamps `deleted_by` on delete) on all parent/content models.
- **File URL accessors** — files are stored as bare filenames; a `get*UrlAttribute()` or `assetUrl()` helper prefixes the public upload dir via `asset()`.
- **Child rows** carry `sort_order`, are returned in display order, and cascade-delete at the DB level (`cascadeOnDelete()`).

Parent → child structure (key groups)

Parent (page)	Children	Join key
HomeAbout	HomeAboutMilestone (<code>milestones</code>)	<code>home_about_id</code>
HomeClientele	HomeClienteleImage (<code>images</code>)	<code>home_clientele_id</code>
ProjectCategory	ProjectListing (<code>listings</code> / <code>projects</code>)	<code>project_category_id</code>
ProjectListing	ProjectDetail (<code>detail</code> , <code>hasOne</code>)	<code>project_listing_id</code>
ProductCategory	ProductListing (<code>productListings</code>)	<code>product_category_id</code>
ContactDetail	ContactSocialLink (<code>socialLinks</code>)	<code>contact_detail_id</code>
AboutUs	AboutUsImage (<code>sectionImages</code>) — section slider	<code>about_us_id</code>
Esg	<code>innovationFeatures</code> , <code>drivingCounts</code> , <code>impacts</code> , <code>wasteFeatures</code>	<code>esg_id</code>
DesignEngineering	<code>features</code> , <code>teamImages</code>	<code>design_engineering_id</code>
ProjectManagement	<code>pointers</code>	<code>project_management_id</code>
Facility	<code>features</code> , <code>counters</code> , <code>galleries</code> , <code>strengths</code> , <code>testingImages</code>	<code>facility_id</code>
Media	MediaDocument (<code>documents</code>)	<code>media_id</code>
DisclosureRow	<code>links</code> , <code>tabs</code> → <code>tabItems</code> (3-level)	<code>disclosure_row_id</code> / <code>disclosure_tab_id</code>
StockExchangeTab	<code>items</code>	<code>stock_exchange_tab_id</code>
Role ↔ Permission	many-to-many	<code>permission_role</code>

Notable special behaviours

- **SeoMeta** — `setUrlPathAttribute()` normalizes the path & auto-derives `page_name`; statics `normalizePath()`, `deriveName()`, `forPath()`.
- **DisclosureRow** — `getRouteKeyName() = 'slug'` (bound by slug on the detail page); `type` enum = `links` | `financial` | `tabs`.
- **DisclosureLink / DisclosureTabItem / StockExchangeItem** — `href` accessor returns the uploaded file URL if present, else the pasted URL.
- **IpoDocument** — `link_url` accessor: external URL takes precedence over a stored PDF; `group` / `subgroup` / `is_group_header` build a hierarchical document list.
- **Banner-on-first-row convention** — several Overview/Investor pages store banner fields on the *first* record of the table (`$banner = Model::first()`).

7. Admin CMS (Backend)

The `$cms()` route helper

All admin modules are registered through one closure in `web.php` that wires a resource controller to its permission middleware:

```
$cms = function (string $name, string $controller, ?array $only = null) {
    $resource = Route::resource($name, $controller);
    if ($only) { $resource->only($only); }
    return $resource
        ->middlewareFor(['index', 'show'], "permission:$name.view")
        ->middlewareFor(['create', 'store'], "permission:$name.create")
        ->middlewareFor(['edit', 'update'], "permission:$name.edit")
        ->middlewareFor('destroy', "permission:$name.delete");
};

$cms('manage-facility', FacilityController::class); // full CRUD
$cms('manage-legal-pages', LegalPageController::class, ['index', 'edit', 'update']); // edit-only
```

Two module shapes

- **Singleton-settings** — one DB row = one page. Registered as `['index', 'update']` or `['index', 'edit', 'update']`; the index is the edit form. Examples: About Us, Careers Details, Legal Pages, Disclosures / Stock Exchange page settings, SEO.
- **Full-CRUD list** — index lists records with create/edit/delete. Typically carry `slug`, `is_active`, `priority`. Examples: Products, Jobs, Board of Directors, Disclosure Items, Stock Exchange Tabs.

Upload handling pattern (per controller)

Each controller defines a `const DIR` (a folder directly under `public/`) and private helpers:

- `storeUpload($file)` — creates the folder if missing, moves the file in, returns the stored filename (only the name is saved in the DB).
- `deleteUpload($name)` — unlinks the file if it exists.
- On **update**, a blank file input keeps the existing file; a new upload deletes the old one. On **destroy**, all files (incl. child-row files) are removed first.

Filename convention (Disclosures & Stock Exchange): attachment uploads keep the *original filename* with spaces replaced by underscores (collisions get `_2`, `_3` ...). Other modules use a `time()_uniqid()` name.

Repeatable child rows ("Add More")

Parent/child relations are synced on save using either a *rebuild* (delete all, recreate from submitted array) or a *diff* (delete only ticked IDs, append new uploads). The UI uses small vanilla-JS partials that clone a template row with an incrementing index and a delegated "remove row" handler (destroying any attached CKEditor instance first).

Standard validation defaults (apply to every new module)

Field type	Rule
Images	<code>file mimes:jpg,jpeg,webp,svg max:2048</code> (2 MB)
Documents	<code>file mimes:pdf,doc,docx,zip,mp3 max:5120</code> (5 MB)
Error display	Inline <code>@error</code> below each field — no top summary block
Success message	No <code>session('message')</code> alert block — the global notify/toastr shows it once
Previews	Live image preview + "Preview" link for uploaded documents (shown only when data exists)

Admin dashboard

`LoginController::dashboard()` → `backend.dashboard` . A read-only overview aggregating counts across every module (projects, products, enquiries, users, content), plus `$projectsByCategory` , recent activities, recent enquiries and recent applications.

Activity Log (audit trail)

- `AppServiceProvider` attaches `AuditObserver` to key content models and listens for auth Login/Logout/Failed events.
- `AuditObserver` logs created/updated/deleted, redacts passwords, records old/new values & changed fields, and recognizes the "soft-delete via update" pattern.
- `ActivityLogger` writes to `activity_logs` (append-only, never throws). Captures user, event, module, IP, method, URL, user-agent.
- `ActivityLogController` (Super Admin only) lists with filters (user, event, module, text, date range) and a detail view.

8. Public Website (Frontend)

All public pages render from `HomeController` ; views live in `resources/views/frontend/` and include the shared partials `head` , `header` , `footer` , `main-js` .

Pages & data sources

Page / Route	Controller method	Main data
/ (Home)	index	banners, about+milestones, clientele, blog, project & product categories, home projects
/about-us	about_us	AboutUs + sectionImages (slider)
/board-of-directors	board_of_directors	directors (banner on first row)
/innovation, /media, /esg, /awards-recognition	respective	parent + children; banner on first row
/design-and-engineering, /facility, /project-management	respective	parent singleton + child collections
/products, /products/{category}	products_*	active categories; products per category (no detail page)
/projects, /projects/{category}	projects_*	categories; active listings
/category/{project}	projects_details	project + detail (404 if mismatch)
/ipo, /ipo/disclaimer, /ipo/disclaimer/confirm	ipo, ipo_disclaimer*	grouped IPO docs; 2-step DRHP gate
/corporate-governance, /annual-report, /investor-resources	respective	grouped / listed documents
/disclosures, /disclosures/{slug}	disclosures, disclosure_tabs	rows by type; tab detail page
/stock-exchange	stock_exchange	tabs + documents
/careers (+ POST /careers/apply)	careers / FormController	jobs; application form
/contact-us (+ POST)	contact_us / FormController	contact details; enquiry form
/privacy-policy, /terms-conditions	legal_page(slug)	LegalPage by slug

Shared components & View Composers

`AppServiceProvider::boot()` registers composers so shared partials always have data:

<code>navCategories</code>	→ header + footer (ProjectCategory, dynamic Projects menu)
<code>navProductCategories</code>	→ header (active ProductCategory, dynamic Products menu)
<code>footerContact</code>	→ footer (latest ContactDetail + socialLinks)
<code>seo</code>	→ head (SeoMeta::forPath(request()->path()))

The **header search** (`#gwsSearch`) debounces input and fetches `route('frontend.search')` which returns JSON across project/product categories & listings. The **footer** renders dynamic contact/social, a dynamic Projects column, and the SPINE HRMS employee-login link.

9. Feature Deep-Dives

9.1 SEO Manager (per-URL meta tags)

Admin module `manage-seo` manages a `seo metas` row per URL (`url_path` , `page_name` , `meta_title` , `meta_description` , `canonical` , `hreflang` , `og_tag` , `twitter_card_tag` , `is_active`). A view composer injects `$seo = SeoMeta::forPath(request()->path())` into `head.blade.php` , which renders title/description/canonical/hreflang/OG/Twitter with safe fallbacks (title → app name, canonical → current URL, default hreflang alternates). `page_name` is auto-derived from the path and is purely an admin-facing label. Seeded from `database/seeder/data/seo_metas.csv` via `SeoMetaSeeder` (81 URLs); OG/Twitter/hreflang are left for the admin to paste raw tag markup.

9.2 Disclosures (SEBI Reg. 46 page)

Admin: *Investors Relations* → *Disclosures* → *Page Settings* (banner + intro heading) and *Disclosure Items* (the rows). Each `DisclosureRow` has a `type` :

- **links** — title + one or more "View" links (URL or uploaded document).
- **financial** — heading + year + named sub-items (a/b/c), each with its own link.
- **tabs** — a "View" link that opens a dedicated page `/disclosures/{slug}` (slug from the title) showing Q1/Q2/Q3 tabs; each tab holds documents (name + attachment); an empty tab shows "Coming Soon".

Row numbers (01, 02...) are **auto-assigned by display order** — there is no manual number/sort field. Links can be a URL (for page links like About Us) or an uploaded attachment (PDF/DOC/DOCX/ZIP/MP3).

9.3 Stock Exchange

Admin: *Investors Relations* → *Stock Exchange* → *Page Settings + Tabs & Documents*. Quarter tabs (Q1/Q2/Q3...) each hold documents (auto-numbered, **name + attachment only** — no URL column). Empty tabs render "Coming Soon". Frontend is tab-switched inline JS.

9.4 Contact & Career Forms

Handled by `FormController` . Shared rules: name regex (letters, starts with a letter, 2–100), phone exactly 10 digits. Both forms require **reCAPTCHA v2** (`App\Rules\Recaptcha` verifies the token server-side against Google). On success the submission is stored (`ContactSubmission` / `CareerApplication`) and **two branded emails** are sent (admin notification + user confirmation) via `BrandedFormMail` ; failures are caught/logged so the user flow never breaks. Careers accepts a resume (pdf/doc/docx, ≤3 MB) stored under `public/uploads/resumes` . Recipients come from `config/forms.php` (`CONTACT_ADMIN_EMAIL` / `CAREERS_ADMIN_EMAIL`). Admins view submissions under *Enquiries*.

9.5 About Us section slider

The About Us "section image" is a **multi-image Swiper slider** (`about_us_images` child table, `sectionImages` relation). Admin uploads one or many images (removable thumbnails); the frontend renders a `.tp-about-slider` Swiper (init already in `main.js`) with a single-image fallback.

9.6 IPO / DRHP disclaimer flow

A two-step legal gate before the DRHP PDF, backed by the single `IpoDrhp` record: `/ipo/disclaimer` (page 1 → Continue) → `/ipo/disclaimer/confirm` (page 2 → I Confirm → DRHP.pdf). IPO documents themselves are grouped (group → subgroup → docs) with optional group-header rows.

10. File Uploads & Storage

Uploads are stored directly under `public/` in per-module folders (NOT the Laravel `storage/` disk), so they are served statically. Each module's controller owns its `DIR` constant. Examples:

- `about/` — About Us images/video/slider
- `board/` — Board of Directors
- `product-categories/` , `product-listings/`
- `project/categories/` , `project/listings/` , `project/details/`
- `facility-uploads/`
- `design-engg/` , `project-mgmt/`
- `esg-uploads/` , `awards-uploads/` , `innovations/`
- `media-uploads/`
- `annual-reports/` , `governance-docs/` , `ipo-docs/` , `investor-uploads/`
- `disclosure-uploads/` , `stock-exchange-uploads/`
- `contact/` , `careers-uploads/` , `uploads/resumes/`
- `legal-uploads/`
- `home/bannerimagevideo/` , `home/aboutmilestones/` , `home/clienteleimages/`

Because uploads live under `public/` , they are **not** in version control / `storage/` . When migrating servers, copy these folders along with the database. Resume uploads under `uploads/resumes/` contain PII.

11. 301 Redirects & .htaccess

There are **two** `.htaccess` files:

- **Root** `.htaccess` (= `public_html` on live) — the one that matters in production. It forces HTTPS, contains the **~40 hard-coded 301 redirects** (old-site → new-site URL map, from the Oct 2026 migration sheet), then rewrites asset requests into `public/` and routes everything else to `server.php`. It also strips trailing slashes, passes through `sitemap.xml` / `robots.txt`, and sets compression + cache headers.
- `public/.htaccess` — the stock Laravel front controller (used only in the standard `public` -as-docroot setup, e.g. local Apache).

The 301 map lives in the **root** file because production serves routes via `server.php` and never passes through `public/.htaccess` for route requests. Redirects are exact-anchored (`^old/?$`) so a category rule does not swallow detail URLs.

12. Third-Party Integrations

Integration	Where	Config
Google Tag Manager	head + header noscript	Container <code>GTM-K5CF8RLQ</code> (hardcoded)
reCAPTCHA v2	Contact + Career forms	<code>services.recaptcha.site_key / secret_key</code> (env)
Elfsight widget	Home "Latest Insights"	App id + loader; overridable via <code>HomeBlog.api_link</code>
SPINE HRMS	Footer Company column	External link to staff login portal
SMTP mail	Form notifications	<code>.env</code> <code>MAIL_*</code> + <code>config/forms.php</code> recipients

Known issue — Elfsight: the free plan enforces a monthly view limit; once exceeded the social-feed widget stops rendering (returns `APP_VIEWS_LIMIT_REACHED`) on all domains. Fix = upgrade the Elfsight plan or replace the widget. This is not a code bug.

reCAPTCHA domains: add the production domain (and `localhost / 127.0.0.1` for local testing) in the reCAPTCHA admin console, or the widget shows a domain warning.

Mail on live: shared hosts often block outbound SMTP to Gmail. Use the domain's own mailbox (`mail.yourdomain.com`) and ensure the recipient domain's cPanel "Email Routing" is set to *Remote* if its email is on Google/Microsoft (otherwise Exim returns `550 No Such User Here`).

13. Seeders & Initial Data

Seeder	Seeds
<code>RolePermissionSeeder</code>	System permissions + roles (superadmin/admin/user); elevates first user to Super Admin.
<code>SeoMetaSeeder</code>	81 SEO rows from <code>data/seo metas.csv</code> (title/description/page name per URL). Preserves admin-entered OG/Twitter.
<code>DisclosureSeeder</code>	Disclosures singleton + ~31 rows (links/financial/tabs) with their children.
<code>StockExchangeSeeder</code>	Stock Exchange singleton + Q1 (11 documents) + empty Q2/Q3.

CMS-module permissions are added by **migrations**, not seeders. The default `DatabaseSeeder` only creates a factory test user — run the real seeders explicitly: `php artisan db:seed --class=RolePermissionSeeder`, etc.

14. Conventions, Gotchas & Known Issues

- **Greedy route ordering** — literal multi-segment routes MUST precede `/{{category}}/{{project}}` (\$4).
- **Single-line @php breaks** — always use multi-line `@php ... @endphp` blocks.
- **Bootstrap .d-block is !important** — it overrides inline `display:none`. Toggle visibility with inline `display:block/none` and avoid pairing `d-block` with an inline `hide`.
- **Page CSS belongs in public/frontend/assets/css/main.css** — not per-page files or inline `<style>` blocks (most page classes already live there).
- **New-module defaults** — image 2 MB (jpg/jpeg/webp/svg), docs 5 MB (pdf/doc/docx/zip/mp3), inline field errors, no duplicate success alert, image + document previews. Apply by default.
- **Banner-on-first-row** — several Overview/Investor pages read banner fields from `Model::first()`.
- **Raw HTML output** — SEO OG/Twitter/hreflang and some headings render via `{!! !!}`; admin input is trusted.
- **Thank-you pages** served via `Route::view` (the equivalent controller methods are unused).
- **Hardcoded IDs** — GTM container and Elfsight app id are in Blade, not env.
- **Self-registration** gives no role — an admin must assign one before the account can do anything.
- **Destructive data ops** — never run whole-table/unfiltered deletes for cleanup; delete only by captured IDs.

15. Deployment & Maintenance Guide

Deploying code changes

```
git pull
composer install --no-dev --optimize-autoloader # if dependencies changed
php artisan migrate --force # apply new migrations
php artisan db:seed --class=SeoMetaSeeder --force # if seed data changed
php artisan config:clear # after ANY .env change
php artisan view:clear
# (optional for perf) php artisan config:cache && php artisan route:cache
```

After editing `.env` on live you MUST run `php artisan config:clear` (or re-run `config:cache`) — cached config ignores `.env` edits, which is a common "why didn't my change take effect" trap.

First-time / fresh install

```
composer install
cp .env.example .env # set APP_KEY, DB_*, MAIL_*, RECAPTCHA_*
php artisan key:generate
php artisan migrate --force
php artisan db:seed --class=RolePermissionSeeder --force
php artisan db:seed --class=SeoMetaSeeder --force
php artisan db:seed --class=DisclosureSeeder --force
php artisan db:seed --class=StockExchangeSeeder --force
```

Server migration checklist

- Copy the database.
- Copy ALL `public/<upload-dir>/` folders (user-uploaded media is not in git).
- Set `.env` (DB, mail, reCAPTCHA keys, admin email recipients) then `config:clear`.
- Upload the correct root `.htaccess` (contains HTTPS + 301 map + rewrite to `public/`).
- Verify reCAPTCHA allowed domains and SMTP/email-routing.

Routine tasks

- **Add an admin user** → Users module, assign a Role.
- **Grant access to a module** → edit the Role, tick the module's view/create/edit/delete permissions.
- **Review audit trail** → Activity Log (Super Admin).
- **Export enquiries** → Enquiries → Contact / Career (view & delete; download resume from the application).

16. Common Admin Tasks (How-To)

Task	Where in admin
Change a page's SEO title/description	SEO Manager → edit the row for that URL path
Add a project	Projects → Listing (set category, slug, image, "show on home") → then Projects → Details for the full page
Add a product	Products → Category, then Products → Listing
Update home hero slides	Home → Banner Details (image or video per slide)
Edit footer address / phone / socials	Contact Details (drives the footer via composer)
Add an IPO / governance / annual-report document	Investors Relations → respective module (PDF upload, group/priority)
Add a disclosure document or quarterly tab	Investors Relations → Disclosures → Disclosure Items (choose row type)
Add a stock-exchange filing	Investors Relations → Stock Exchange → Tabs & Documents (name + attachment)
Post a job	Careers → Job posting
Add About Us slider images	Overview → About Us → Section Images (multi-upload)
Edit Privacy Policy / Terms	Legal Pages

17. Appendix

A. Admin modules & route names

Each has `.view/.create/.edit/.delete` permissions unless restricted.

- Home: `banner-details`, `home-about-details`, `home-clientele`, `home-blog-details`
- Overview: `manage-about-us`, `manage-board-of-directors`, `manage-innovation`, `manage-esg`, `manage-media`, `manage-awards-category`, `manage-awards-recognition`
- Products: `manage-product-category`, `manage-product-list`
- Projects: `manage-project-category`, `manage-project-listing` (+toggle-home), `manage-project-details`
- Infrastructure: `manage-design-engg`, `manage-facility`, `manage-project-management`
- Investors: `manage-ipo`, `manage-ipo-drhp`, `manage-corporate-governance`, `manage-annual-report`, `manage-investor-resource`, `manage-disclosures` (index/update), `manage-disclosure-items`, `manage-stock-exchange` (index/update), `manage-stock-exchange-tabs`
- Careers: `manage-careers-details`, `manage-jobs`
- Contact: `manage-contact-details`
- Enquiries: `manage-contact-enquiries` (view/delete), `manage-career-applications` (view/delete)
- Legal: `manage-legal-pages` (view/edit)
- SEO: `manage-seo`
- System: `roles.*`, `users.*`, `permissions.view/assign`, `dashboard.view`

B. Key config files

<code>routes/web.php</code>	All routes + the <code>\$cms()</code> helper
<code>bootstrap/app.php</code>	Middleware aliases, guest/auth redirects
<code>config/services.php</code>	reCAPTCHA site/secret keys
<code>config/forms.php</code>	Contact/Career admin email recipients
<code>config/mail.php</code> + <code>.env</code>	SMTP configuration
<code>app/Providers/AppServiceProvider.php</code>	Audit observers + view composers

C. Environment variables to set

```
APP_NAME, APP_ENV, APP_KEY, APP_URL, APP_DEBUG
DB_CONNECTION, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD
MAIL_MAILER, MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD,
MAIL_ENCRYPTION, MAIL_FROM_ADDRESS, MAIL_FROM_NAME
CONTACT_ADMIN_EMAIL, CAREERS_ADMIN_EMAIL           # form recipients
RECAPTCHA_SITE_KEY, RECAPTCHA_SECRET_KEY           # reCAPTCHA v2
```

Start here as a new developer: read `routes/web.php` (the `$cms()` helper + route order), then one singleton module end-to-end (e.g. `FacilityController` + its Blade views + the `Facility` model & children). That single vertical slice demonstrates the pattern used by every other module.